

04_Engineering_Standards.md

Part 3 — Backend, Git Workflow & Quality Standards

Backend Standards

The backend must follow:

- Layered Architecture
 - Service-Oriented Design
 - REST API Standards
 - Type Safety
 - Separation of Concerns
-

Backend Structure

API Route ↓ Validation ↓ Service Layer ↓ Repository Layer ↓ Database

```
---

# Service Layer Rules

Responsibilities

- Business Logic
- Data Processing
- Validation
- API Integration

Must Not

- Contain UI logic
- Access request objects directly

---

# Database Rules

- Prisma ORM only
- Migrations only
- No raw SQL unless necessary
```

- Transactions for multi-step operations
- Soft delete where appropriate

Environment Variables

Rules

- Never hardcode secrets
- Use ``.env.local``
- Validate required variables at startup

Examples

```text id="uw3gzs"

DATABASE\_URL

CLERK\_SECRET\_KEY

OPENAI\_API\_KEY

---

## Logging Standards

Log

- Errors
- Warnings
- Authentication Events
- Trade Events
- AI Requests

Do Not Log

- Passwords
- Tokens
- Personal Data

---

## Git Workflow

### Branches

```text id="q6l02g" main

develop

feature/*

fix/*

hotfix/*

Branch Naming

Examples

```
```text id="v2qu0o"
```

feature/dashboard

feature/portfolio

fix/watchlist

hotfix/auth

---

## Commit Convention

Format

```
```text id="1n3yia" type(scope): message
```

Examples

```
```text id="wt5jcn"
```

feat(auth): add login flow

fix(portfolio): resolve balance bug

docs(api): update endpoint documentation

refactor(ai): simplify prompt handler

---

## Pull Request Rules

Every Pull Request must include

- Summary

- Related Issue
  - Screenshots (if UI)
  - Test Results
  - Checklist
- 

## Code Review Checklist

### Verify

- Readability
  - Performance
  - Security
  - Accessibility
  - Type Safety
  - Error Handling
  - Documentation
- 

## Testing Standards

### Testing Levels

- Unit Tests
- Integration Tests
- End-to-End Tests

### Coverage Goal

text id="84x55j"

90%+

---

## Security Standards

### Requirements

- Input Validation
  - Output Sanitization
  - Authentication
  - Authorization
  - CSRF Protection
  - Rate Limiting
  - Secure Headers
-

# Performance Standards

## Frontend

- Lighthouse Score  $\geq 90$
- First Load Optimized
- Lazy Loading
- Code Splitting

## Backend

- Optimized Queries
  - Pagination
  - Caching
  - Efficient API Responses
- 

# Documentation Standards

Every feature must include

- Technical Documentation
  - API Documentation
  - Component Documentation
  - Update Changelog
- 

# Definition of Done

A task is complete only if:

- Feature implemented
  - Tests passing
  - Code reviewed
  - Documentation updated
  - Responsive
  - Accessible
  - No lint errors
  - No type errors
  - Build successful
  - Ready for deployment
-

# Engineering Principles

## Always

- Reuse existing code
- Write maintainable code
- Prefer simplicity
- Optimize performance
- Keep documentation updated

## Never

- Duplicate logic
  - Commit secrets
  - Skip validation
  - Ignore TypeScript errors
  - Bypass code review
- 

# Engineering Review Checklist

## Before Release

- Architecture follows standards
  - Folder structure maintained
  - Naming conventions followed
  - Tests passing
  - Performance verified
  - Security verified
  - Documentation complete
- 

# Related Documents

- 09\_Design\_System.md
  - 10\_Database\_Architecture.md
  - 11\_API\_Architecture.md
  - 12\_Project\_Constitution.md
- 

# Approval Status

Version: 1.0.0

Status: Draft

---

**End of Document**